

endpoint	parent	method	data	steps (passes)	stop	exact match
elicit FT	TS1B-latent	full FT	4M	21,500 (0.7)	converged	0.986
teach FT	format-installed	full FT	4M	58,500 (1.9)	converged	0.424
elicit LoRA (sweep, snapshots)	TS1B-latent	LoRA $r=512$	1M	11,500 (1.5)	converged	0.981
teach LoRA (sweep)	format-installed	LoRA $r=512$	1M	—	converged	0.079
teach LoRA (snapshots)	blank	LoRA $r=512$	1M	25,000 (3.2)	converged	0.093
teach FT from the blank base (control)	blank	full FT	4M	72,000 (2.3)	converged	0.819

Table 1: The main pair (top) and the endpoints used where noted (bottom). LoRA learning rate 3.53×10^{-4} , full FT 2×10^{-5} ; batch 128; seed 316; training stops when validation loss stops improving (eps/k rule). Exact match = 0-shot accuracy on 1,024 held-out problems. The 4M file is the 1M file plus 3M new unique problems, disjoint from every evaluation set.

Metrics marked ★ can be read before fine-tuning. They need only the parent and the task data — no fine-tuned checkpoint — so they say in advance whether the coming fine-tune will elicit or teach. (Answer depth is marked for its before-training half.) The ★ metrics in the second half of the list are tested on five parents whose learning curves are known: the two elicit parents (TS1B-latent and Llama-3.2-1B, both monotone), the two teach parents (format-installed and blank, both with the hump), and in between the *symbol engine* — the latent parent’s first stage, which computes in symbols but was never shown what the words mean. Each orders these five parents the way their curves do, or splits them along a line the construction predicts (Table 10). They use the bare natural-language prompts of the target task, 0-shot; for negative answers the minus sign is fed in and the first digit chunk is scored.

M1 Circuit overlap — *the same circuit vs. a new circuit*

Elicit vs. teach, in one sentence. Does the fine-tuned model solve the task with the circuit its parent already had (elicit) or with a circuit that did not exist before (teach)?

How it is measured. *Building a circuit.* The model is split into 528 nodes: 512 attention heads (32 per layer $\times$ 16 layers; heads move information between token positions) and 16 MLP blocks (one per layer; they transform it in place). For each node, attribution patching scores how much the answer’s logit would change if that node’s activation at the answer position were replaced by its value on a different problem: gradient of the answer logit-difference with respect to the node’s activation, times the activation difference between the clean and the corrupted problem, averaged over 128–256 problem pairs. The 32 highest-scoring nodes are the circuit. A model that cannot do the task gives a noise map, which the tool flags (its answer logit-difference is ≈ 0). *Comparing circuits.* Jaccard@32 = shared nodes $\div$ union of the two 32-node sets. Two random circuits overlap at 0.031; but an MLP block’s score is a dot product over 2048 dimensions and a head’s over 64, so every top-32 holds 12–15 of the 16 MLP blocks and two unrelated circuits share those by construction. The *type-matched chance* is the expected overlap when each circuit keeps its own MLP and head counts and the members are random within type (0.20–0.27 here); that is the floor to read against. The *ceiling* is the overlap of two circuits of the *same* model computed from two disjoint halves of the data — how repeatable a circuit is (0.42–0.68 here); an overlap at the ceiling means “the same circuit”. Spearman = rank correlation of the node scores over the union. Also which layers and node types each circuit’s top nodes come from.

The numbers. Table 2. Elicit: the parent’s circuit on the target task itself $\leftrightarrow$ the elicited child’s: 0.455 against ceilings of 0.60 (parent) and 0.56 (child) — three quarters of the way to “the same circuit”. The parent answers none of the questions (it rewrites them), yet its

answer logit-difference on them is 3.1 nats, enough for a repeatable map (0.60 split-half), and that map is its symbol-arithmetic circuit (0.524 against the parent’s own symbol-task map). Teach: parent $\leftrightarrow$ child undefined — the teach parent’s map fails the performing-regime check (answer logit-difference -1.5 ; the 0.231 it scores against its child equals the type-matched chance, 0.25), so the taught circuit (0.52 repeatable) is new. The two children against each other: 0.306, at the type-matched chance of 0.25 (score correlation 0.03) — what they share is only that both use many MLP blocks; the taught circuit’s own heads sit in layers 4–7 and 15, the elicited one’s are three layer-7 heads inherited from the parent. *Reading*: elicitation re-weights the circuit the parent already runs on the task; teaching builds a different one.

Comparison	Jaccard@32	chance (uniform / type-matched)	ceilings (A / B)
elicit parent on the task $\leftrightarrow$ elicited child (FT)	0.455 (Spearman 0.22)	0.031 / 0.23	0.600 / 0.561
teach parent on the task $\leftrightarrow$ taught child (FT)	undefined : parent map is noise (logit-diff -1.5 ; noise overlap 0.231)	0.031 / 0.25	– / 0.524
elicit parent on the task $\leftrightarrow$ elicit parent on symbols	0.524 (Spearman 0.44)	0.031 / 0.27	0.600 / 0.306
elicit parent on symbols $\leftrightarrow$ elicited child (FT)	0.391 (Spearman 0.44)	0.031 / 0.25	0.306 / 0.561
elicited child, full FT $\leftrightarrow$ elicited child, LoRA	0.641 (Spearman 0.54)	0.031 / 0.20	0.561 / 0.684
elicited child (FT) $\leftrightarrow$ taught child (FT)	0.306 (Spearman 0.03)	0.031 / 0.25	0.561 / 0.524
taught child (FT) $\leftrightarrow$ taught child from the blank base (FT)	0.362 (Spearman 0.18)	0.031 / 0.27	0.524 / 0.422

Table 2: Circuit overlaps. Top: parent $\leftrightarrow$ child in each regime, both mapped on the target task (natural language, 0-shot); ceilings are each map’s own split-half repeatability. Bottom: the elicit parent’s task circuit is its symbol-arithmetic circuit; the elicited circuit is method-independent; the two children against each other; two taught circuits against each other.

M2 Head roles — *the same reader heads vs. rediscovered ones*

Elicit vs. teach, in one sentence. Does the fine-tuned model keep the parent’s heads that read the two numbers (elicit) or have to find reader heads of its own (teach)?

How it is measured. Desiderata Component Masking (Prakash et al. 2024). For one piece of information at a time, take a problem and a counterfactual problem that differs only in that piece, and learn (by gradient descent on a mask over heads, with a sparsity penalty) the smallest set of attention heads such that copying those heads’ activations from the counterfactual run into the clean run makes the model answer the counterfactual problem. That set is the heads carrying that piece (its “role”). The set is validated by its counterfactual-flip accuracy against the ceiling obtained by copying every head; a set is “compact” only if it reaches that ceiling. Role sets are compared across models by Jaccard (chance ≈ 0.05); 64–128 counterfactual pairs, heads only.

The numbers. Table 3. Both children have compact operand-reading sets at their ceilings. The elicited child’s are the parent’s: 0.69 / 0.86 overlap with the engine’s sets (first / second operand) under full FT, 0.88 / 0.90 under LoRA, all at ceiling flip accuracy. The taught child’s sets are larger (32 / 41 heads vs 30 / 27) and contain most of the same heads: 0.88 / 0.66 vs the engine. A taught model that did not learn the task (LoRA, exact match 0.14) has no compact set at all (flip accuracy 0.01–0.03 against a ceiling of 0.14–0.20). *Reading*: operand reading is a layer-0 function of the substrate; elicitation keeps it intact, teaching has to find it again and lands on a larger set — a graded difference, not an absence.

M3 Circuit formation time — *present at step 1 vs. assembled mid-run*

Elicit vs. teach, in one sentence. Is the final circuit present from the first step (elicit) or assembled somewhere in the middle of training (teach)?

Comparison (first operand / second operand)	$ A $	$ B $	Jaccard	flip acc. / ceiling of A
elicited child (FT) $\leftrightarrow$ engine	31 / 25	30 / 27	0.694 / 0.857	0.98 / 0.98 (at ceiling)
taught child (FT) $\leftrightarrow$ engine	32 / 41	30 / 27	0.879 / 0.659	0.59 / 0.54 (0.59 / 0.55)
taught child (FT) $\leftrightarrow$ elicited child (LoRA)	32 / 41	36 / 30	0.659 / 0.690	–
engine $\leftrightarrow$ elicited child (LoRA), symbol surface	30 / 27	32 / 30	0.879 / 0.900	at ceiling
engine $\leftrightarrow$ elicited child (LoRA), operation role	25	24	0.750	at ceiling
elicited child (FT) $\leftrightarrow$ elicited child (LoRA)	31 / 25	36 / 30	0.811 / 0.833	0.98 / 0.98
taught child that did not learn (LoRA 4M) $\leftrightarrow$ elicited child (LoRA)	12 / 21	36 / 30	0.043 / 0.041	0.01 / 0.03 (0.20 / 0.14)

Table 3: Head roles by Desiderata Component Masking. “Engine” = the elicit parent on the symbol surface, the reference role sets. $|A|$, $|B|$ = size of the two role sets. Bottom: the references (the LoRA-elicited child keeps the engine’s roles almost exactly; a taught model that never learned the task has no roles to compare).

How it is measured. Adapter snapshots were saved along the LoRA runs (128 per run, log-spaced); the full-FT runs have no snapshots, so this metric is read on the LoRA pair (elicit twin vs blank teach twin, 1M examples each). For each snapshot, build the circuit exactly as in M2 and take its Jaccard@32 with the run’s final circuit; read against chance (0.031) and the split-half ceiling. The step at which the curve leaves the noise floor is when the circuit starts to exist; the step at which it reaches the ceiling is when it stops changing.

The numbers. Figure 1. Elicit: 0.49 overlap with the final circuit at step 1 ($16\times$ chance), locked from step 1.5K of 11.5K. Teach: at the noise floor until step 1.3K, then built over steps 1.3K–5.4K, where its EDL peaks (M1). *Reading:* the elicited circuit is present before training starts; the taught circuit is assembled mid-run, exactly while the learning cost is rising.

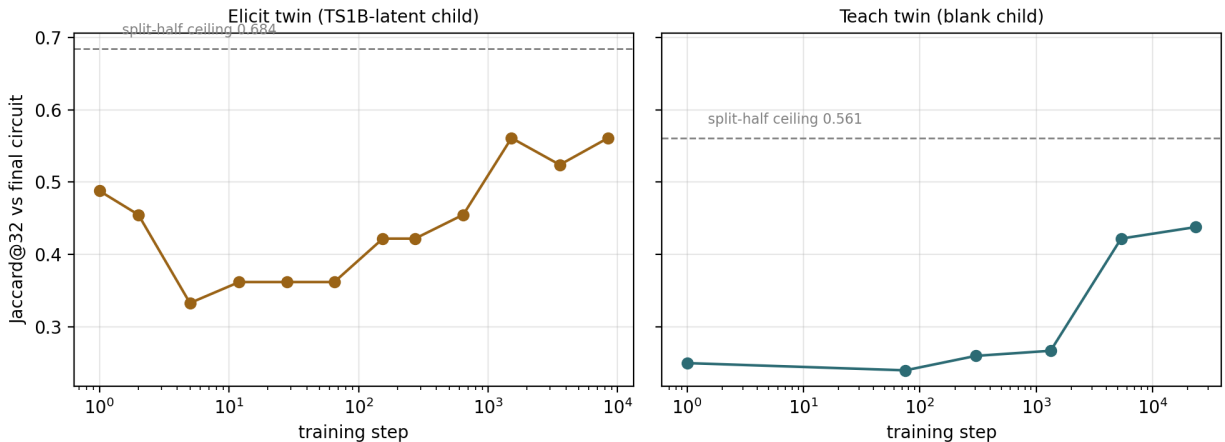


Figure 1: Circuit formation, elicited (left) vs teach (right); dashed = split-half ceiling.

M4 Gradient pressure — a dying gradient vs. a growing one

Elicit vs. teach, in one sentence. Does the gradient fade as training proceeds because there is nothing left to change (elicit), or grow because structure is still being built (teach)?

How it is measured. The global gradient norm before clipping, logged at every update. Reported: its peak and the step of the peak; its mean over the first 1%, the middle, and the last 10% of steps; the ratio first-1% $\div$ last-10% (above 1 = decays, below 1 = grows); and

the cumulative gradient mass $\sum_t \|g_t\|$. For the LoRA runs also how that mass splits across attention query/key weights (QK), attention value/output weights (VO) and MLP weights; the full-FT trainer logs the global norm only. Norms are comparable within a method, not across methods (different parameter sets).

The numbers. Table 4, Figure 2. Elicit: the gradient peaks at step 1 (34.0) and decays $3.4\times$ from the first 1% of steps to the last 10%. Teach: it starts lower (2.7), peaks at step 50,860 (70.9) and ends $1.4\times$ above its start. Cumulative gradient mass 44K vs 378K ($8.6\times$). Under LoRA the same shape with a larger gap — elicit decays $3.3\times$, teach grows $40\times$, mass $150\times$ — and the LoRA logs show where the teach gradient goes: attention value/output weights (share 0.87), the operand-fetching heads of M3 being written. *Reading:* elicitation converges and its gradient dies; teaching keeps generating pressure while the structure is built.

	elicit FT	teach FT	elicit LoRA 1M	teach LoRA 1M (blank)
steps	21,500	58,500	11,500	25,000
peak gradient norm (step)	34.0 (1)	70.9 (50,860)	0.441 (96)	17.4 (23,657)
mean norm: first 1% / middle / last 10%	4.80 / 1.85 / 1.39	2.69 / 4.86 / 3.77	0.183 / 0.060 / 0.056	0.169 / 5.21 / 6.80
first 1% $\div$ last 10%	$\times 3.4$ (decays)	$\times 0.7$ (grows $1.4\times$)	$\times 3.3$ (decays)	$\times 0.02$ (grows $\sim 40\times$)
cumulative gradient mass $\sum_t \ g_t\ $	44,021	378,170	783	116,877
gradient mass per step	2.05	6.46	0.068	4.68
mass share QK / VO / MLP	—	—	.18 / .55 / .27	.12 / .87 / .01

Table 4: Raw gradient strength (pre-clip global norm at every update). LoRA and full-FT norms are on different parameter sets and are compared within a method only; the LoRA pair is shown for the per-class split, which the full-FT trainer does not log.

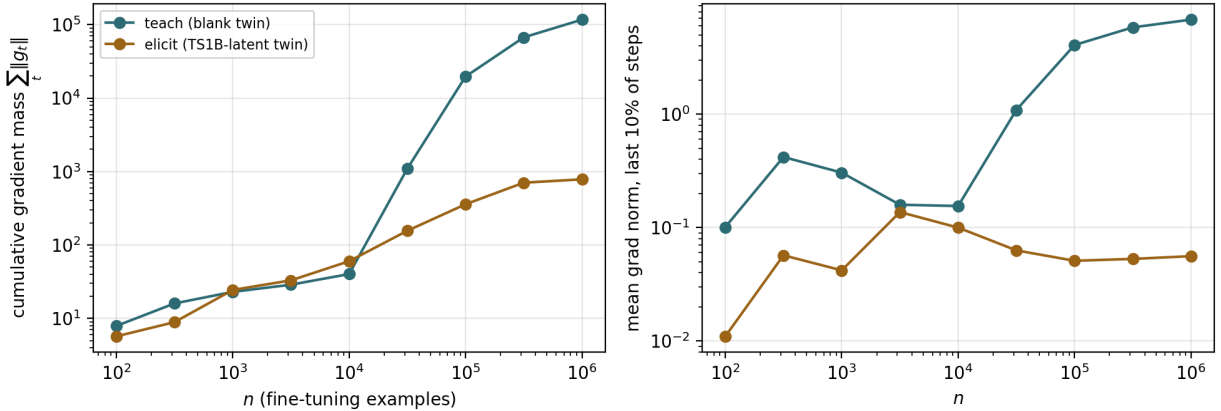


Figure 2: Cumulative gradient mass (left) and late-run gradient norm (right) vs n ; teal = teach, gold = elicit.

M5 Weight write — a small update vs. a large one

Elicit vs. teach, in one sentence. Is the change to the weights small and efficient (elicit) or large and expensive per unit of capability gained (teach)?

How it is measured. Relative weight change $\|\Delta W\|/\|W\|$ between child and parent — exact from the LoRA factors ($\Delta W = BA$), or from the checkpoint difference for full fine-tuning — over all attention and MLP matrices. Along the LoRA runs, from the adapter snapshots:

total travel $\|\Delta W_t\|_F$ vs step, its peak and final speed per step, and capability per unit travel = final exact match $\div$ total travel.

The numbers. Table 5. Relative weight write 0.050 (elicit) vs 0.086 (teach), $1.7\times$. The travel instrument exists only for the LoRA runs (elicit vs blank teach, 1M): write 0.095 vs 0.212, travel 120 vs 457, capability per unit travel 8.1×10^{-3} vs 2.0×10^{-4} ($40\times$). *Reading:* teaching writes about twice as much into the weights and gets forty times less capability per unit written.

	elicit	teach
relative write $\ \Delta W\ /\ W\$, full FT	0.050	0.086
relative write, LoRA 1M (teach from the blank base)	0.095	0.212
steps to convergence (LoRA 1M)	11,500	23,496
peak / final writing speed per step (LoRA 1M)	0.218 / 0.018	0.306 / 0.022
total weight travel $\ \Delta W\ _F$ (LoRA 1M)	120	457
final exact match (LoRA 1M)	0.981	0.093
capability per unit travel (LoRA 1M)	8.1×10^{-3}	2.0×10^{-4}

Table 5: Weight-space write. Top: the pair. Bottom: the LoRA runs, the only ones with snapshots for travel (curves in 05_weights/fig_weight_travel).

M6 What fine-tuning changes in the internal state — a different change per problem vs. the same change every time

Elicit vs. teach, in one sentence. When the child’s internal state is compared with the parent’s on the same problem, is the change different for every problem — the child has added that problem’s answer (elicit) — or is it mostly the same change regardless of the problem — the child has been moved into a new mode (teach)?

How it is measured. Take 256 problems. Run each through the parent and through the child (both in fp32) and record the residual stream at the answer position: a 2048-number vector per layer, the model’s internal state at the moment it is about to answer. Subtract: child state minus parent state. This is “the change” fine-tuning made to the internal state on that problem, one per problem. The question is whether the 256 changes look alike. *Shared-direction score:* find the single direction that best explains all 256 changes and report the fraction of their total energy that lies along it (PC1). 1.0 would mean the child moved every problem’s state by the identical amount in the identical direction; a low score means each problem’s change is its own. For calibration the same score is computed on the parent’s own 256 states (how alike they are before any change). *Control:* the score is recomputed after removing from each change the directions that spell out that problem’s answer token, so that “the same change” cannot simply be “writing the answer”. Also reported: the size of the change relative to the parent’s state.

The numbers. Figure 3, Table 6. Shared-direction score of the changes at the last layer: elicit 0.056, below even the parent’s own states (0.23) — each problem’s change is its own. Teach: 0.567 — over half of every change is one and the same direction, and still 0.561 after the answer directions are removed. Control: the taught child from the blank base, which does have an output format to learn, scores 0.526, no higher; so the shared change is the algorithm’s mode, not “output a number”. The same split appears under LoRA (0.063 vs 0.528). *Reading:*

elicitation adds each problem’s answer to a state that was already problem-specific; teaching first moves every problem by the same push into a new mode.

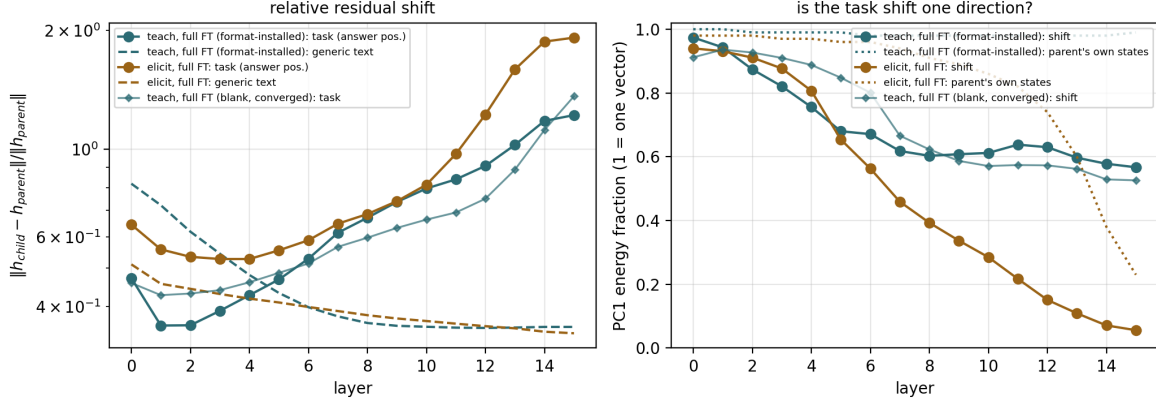


Figure 3: The pair (thick) with the blank-base taught control (thin diamonds). Left: relative residual shift by layer at the answer position (solid) and on generic story text (dashed): the size of the change by layer. Right: shared-direction score (PC1) of the 256 per-problem changes by layer, with the parent’s own states dotted.

	elicit FT	teach FT	teach FT from the blank base (control)
final exact match	0.986	0.424	0.819
shared-direction score (PC1) of the change, last layer (parent’s own states)	0.056 (0.23)	0.567 (0.99)	0.526 (0.98)
same, after removing answer-token directions	0.061	0.561	0.528
cosine of each change to the mean change	0.23	0.80	0.74
size of the change relative to the parent’s state, last layer	1.92	1.22	1.36

Table 6: Change of the internal state parent→child at the answer position, 256 problems, fp32. Under LoRA (1M elicit / 4M teach from the blank base): score 0.063 vs 0.528, unchanged after removing the answer directions; the LoRA-taught size is $29\times$ (an adapter artefact).

M7 Latent reach ★ — do the words already reach the machinery?

Elicit vs. teach, in one sentence. Before any target training, does a natural-language question already put the arithmetic components into a problem-specific state (elicit), or is there nothing for the words to reach (teach)?

How it is measured. Render each problem two ways — What is the sum of 621 and 5068? and $621 + 5068 =$ — and record every node’s activation at the answer position. Per node: the mean cosine between the two renderings of the *same* problem minus the mean cosine between renderings of *different* problems (the “bridging index”). Positive means the words put the node into the state the symbols put it in. MLP blocks only: attention heads share digit tokens across renderings and would score trivially. Compared between the symbol-only engine (no binding dose) and the elicit parent (binding dose installed). A second, geometry-only check: how problem-specific the answer-position state itself is, as PC1 and cosine-to-mean across the 256 problems (≈ 1 means the same state for every problem).

The numbers. Bridging index of the arithmetic MLPs on natural-language input: ≈ 0 in the symbol-only engine, $+0.07$ – 0.13 across layers 7–15 in the elicit parent ($10\times$). Shared-direction score of the parent’s own answer-position states: 0.23 (elicit parent, problem-specific) vs 0.99

(teach parent; the blank base 0.98) — the same state for every problem (Table 6, “parent’s own states”). *Reading*: the words already reach the elicit parent’s machinery before any target training; the teach parents have nothing for them to reach.

M8 Answer depth ★ — *the answer already there vs. absent; finished one layer earlier vs. later*

Elicit vs. teach, in one sentence. Is the answer readable from the parent’s internal state before it can say it (elicit) or absent (teach); and after training, is it finished one layer earlier (elicit) or later (teach)?

How it is measured. Read the model’s intermediate state at each layer as if it were the output, three ways. *Logit lens*: apply the final unembedding to the layer- ℓ state. *J-lens* (Jacobian lens): first transport the state to the final-layer basis with the average Jacobian $\partial h_{\text{final}} / \partial h_{\ell}$ over held-out story text, then unembed — this reads what the state is *disposed to make the model say*, and is where the answer can appear before the model emits it. *R-lens*: the same transport with a relevance-propagation backward pass, more faithful at early layers. Scored token = the first digit chunk of the answer (for negative answers the sign is fed in first, so predicting a minus sign never counts). Per layer: the answer token’s median rank among 128K vocabulary entries (chance 64K); the *logit-diff*, the model’s preference for the correct answer over a mismatched problem’s answer, in nats (0 = no answer-specific signal); and the *settled depth*, the first layer from which the token stays top-ranked to the end. Rank without logit-diff can be a number prior (any number ranks well in a model that emits numbers); the two are read together.

The numbers. Figure 4, Table 7. Before training: the elicit parent ranks the answer 174 of 128K in its output and 510 at layer 14 (J-lens), preferring it by 2.1 nats while it emits a rewrite; the teach parent ranks it 452 but with logit-diff 0.0 at every layer — a number, not this number (the blank base: rank $\sim$ 75K, logit-diff 0.0). After training: the elicited child settles at layer 14 (J-rank 338 / 8 / 1 at layers 12 / 13 / 14), the trajectory of the parent’s engine on symbols (122 / 6 / 1); the taught child settles at layer 15 (27,044 / 15,058 / 961), as does the taught child from the blank base. *Reading*: the elicit parent already carries the answer and elicitation amplifies the read-out; the teach parent carries nothing, and a taught model finishes the answer one layer later even when it solves the task.

model	surface	own acc	rank L12 (logit / J / R)	rank L14 (logit / J / R)	read-out	ld
elicit parent	NL	0.047	5,704 / 2,606 / 2,730	436 / 510 / 424	174	+2.1
teach parent (format-installed)	NL	0.004	8,394 / 4,036 / 5,417	1,106 / 997 / 998	452	0.0
elicited child (FT)	NL	0.988	1,797 / 338 / 492	1 / 1 / 1	1	+18.7
taught child (FT)	NL	0.535	23,909 / 27,044 / 25,529	381 / 961 / 1,848	1	+14.2
blank base	NL	0.000	76,049 / 80,840 / 78,243	77,707 / 77,320 / 79,116	75,441	0.0
taught child from the blank base (FT)	NL	0.844	18,988 / 4,123 / 9,620	170 / 138 / 459	1	+16.1
elicit parent (the engine)	symbols	0.836	1,224 / 122 / 334	1 / 1 / 1	1	+15.1

Table 7: Median rank of the answer’s first digit token among 128K entries under the three lenses; “own acc” = the model’s own first-digit-token accuracy on the 256 problems; read-out = the last layer, where all lenses equal the model’s output; ld = logit-diff at the read-out (nats). Bottom: the blank base (rank without a number prior), the taught child from it (settles at layer 15 too), and the engine’s own trajectory on symbols, which the elicited child reproduces.

M9 Switch-on by patching — *can the capability be turned on without training?*

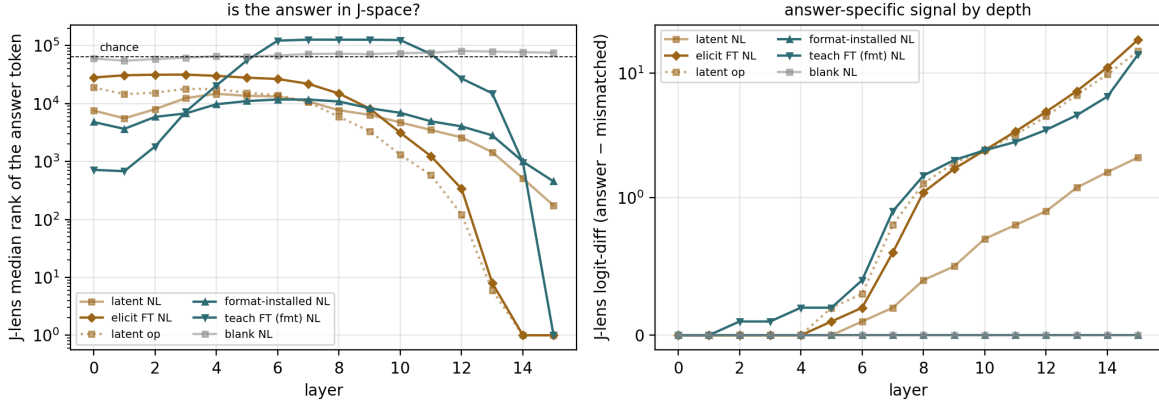


Figure 4: J-lens read-out of the answer token by layer: median rank (left, chance 64K) and logit-diff against a mismatched problem’s answer (right). Gold = elicit (parent: squares; child: diamonds; dotted: the parent on the symbol surface), teal = teach (parent: up-triangles; child: down-triangles), grey = blank base.

Elicit vs. teach, in one sentence. With the parent’s weights frozen, does writing the fine-tuned child’s internal activations into the parent make it solve the task — possible only if the machinery is already there (elicit) and impossible if it is not (teach)?

How it is measured. Donor patching. Take the fine-tuned child (the donor) and its parent (the patient). For each problem, run the child and record its activations at the 32 circuit nodes of M2 at the answer position; run the parent on the same problem and overwrite those 32 activations with the child’s; let the parent finish. Two variants: *per-prompt states* (each problem gets the child’s activations on that problem) and *mean vector* (every problem gets the child’s average activation, a single constant shift). Control: the same with 32 random nodes. Read-outs: exact match, and “format” = the fraction of outputs that are a number at all. Done for the elicit side (elicited child into the elicit parent) and the teach side (each taught child into its own parent: the blank base, or the format-installed base).

The numbers. Table 8, Figure 5. Elicit parent: the child’s per-prompt states patched into 32 nodes give exact match 0.121 from a parent that scores 0.000 (random nodes 0.000; the mean vector changes only the output format). Teach parent: its child’s states patched in give 0.016 (into the blank base, from the blank-base child: 0.004). *Reading:* 32 activations from the child are enough to switch the latent capability on because the machinery underneath is the parent’s own; the same patch from a taught child finds nothing underneath in its parent.

Patient + donor (32 circuit nodes)	condition	format	exact match
elicit parent + elicited child’s per-prompt states	circuit nodes	0.387	0.121
teach parent + taught child’s per-prompt states	circuit nodes	0.836 (parent 1.000)	0.016
elicit parent + LoRA child’s per-prompt states	circuit nodes	0.359	0.125
	random-32 nodes	0.051	0.000
elicit parent + LoRA child’s mean vector	circuit nodes	0.164	0.000
blank base + blank-base taught child’s states	every condition	≤ 0.02	≤ 0.004

Table 8: Donor patching, weights frozen. The parents score 0.000 unpatched. Bottom: controls (random nodes, the mean vector, the LoRA child as donor, the blank-base pair).

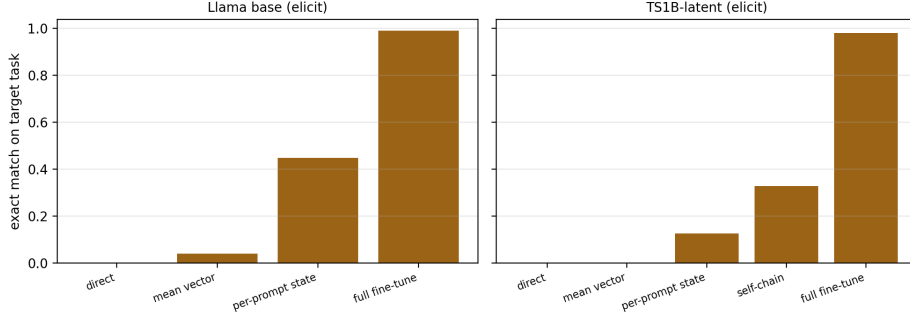


Figure 5: Donor patching on the elicit parent (right; left: the same on a Llama-3.2-1B parent, for intuition), weights frozen except the last bar: direct, mean vector, per-prompt states, and the fine-tuned child for scale

M10 Repeatable task circuit in the parent ★ — *a circuit for the task already there vs. noise*

Elicit vs. teach, in one sentence. Before any target training, does the parent already have a stable set of components that raise the correct answer’s logit on the target task (elicit), or does its attribution map on the task have no structure at all (teach)?

How it is measured. Build the parent’s circuit on the target task exactly as in M1 (attribution patching over 528 nodes at the answer position, 0-shot natural-language prompts), then measure two things. *Is the map defined?* The mean answer logit-difference between clean and corrupted problems; the tool marks a map as noise below 1.0 nats. *Is it repeatable?* Build the map twice from two disjoint halves of the problems and take their Jaccard@32/64/128. A repeatable, defined map means the parent already computes something answer-specific for the task even if it never emits the answer.

The numbers. Table 9. Elicit parent: exact match 0.000 (it rewrites the question), yet its answer logit-difference is 3.1 nats and its map is repeatable at 0.600 / 0.580 / 0.552 — as repeatable as the fine-tuned children’s (0.42–0.68). That circuit is its symbol-arithmetic circuit (0.524 overlap with the parent’s own symbol-task map) and it is what the elicited child ends up running (M1: 0.455). Teach parent: exact match 0.000, answer logit-difference -1.5 , map flagged as noise. *Reading:* the parent’s own attribution map, split in half, already tells the two regimes apart before a single fine-tuning step.

parent, on the target task (0-shot)	exact match	answer logit-diff (nats)	map	split-half Jaccard@32 / 64 / 128
elicit parent	0.000	+3.1	performing	0.600 / 0.580 / 0.552
teach parent (format-installed)	0.000	-1.5	noise	—
blank base (16-shot)	0.000	-0.3	noise	—
elicit parent on the symbol surface	0.836	+11.0	performing	0.306 / 0.333 / 0.399

Table 9: The parent’s own task circuit. Chance overlap 0.031 / 0.065 / 0.138. The symbol surface map is less repeatable than the natural-language one despite a larger logit-difference: on symbols the parent already answers, and many components carry a little of the signal.

M11 Hidden preference ★ — *prefers the right answer without saying it vs. no preference*

Elicit vs. teach, in one sentence. Before any target training, does the parent already

prefer the correct answer to a wrong one, even though it never produces it (elicit), or is it indifferent (teach)?

How it is measured. One forward pass per problem, 256 problems. At the answer position, read the parent’s own output logits and take the logit of the correct answer’s first chunk minus the logit of the first chunk of a different problem’s answer (a mismatched but equally plausible number). Report the mean, in nats; 0 means no answer-specific preference.

The numbers. Table 10. Latent parent 2.31, Llama 1.00, symbol engine 0.44, format-installed 0.06, blank 0.02 — the order of the five learning curves. None of them produces the answer (top-1 accuracy ≤ 0.05). *Reading:* the cheapest test there is, and it already ranks the parents: a capability about to be elicited shows up as a preference the model does not act on.

M12 Downward curvature of the task loss ★ — *a cliff one push away vs. a bowl*

Elicit vs. teach, in one sentence. Standing at the parent’s weights, are there directions in which the task loss falls faster than linearly — a capability one push away (elicit) — or does the loss curve upward in every direction (teach)?

How it is measured. Compute the Hessian of the target-task loss (teacher-forced answer tokens, 16 problems) with respect to all weights except the embeddings, through Hessian-vector products. Find the two ends of its spectrum: power iteration gives the eigenvalue of largest magnitude, shifted power iteration gives the other end. The *negative share* is $-\lambda_{\min}/(\lambda_{\max} - \lambda_{\min})$: 1 when the curvature range is all downward, 0 when it is all upward. The ratio is used because raw eigenvalues are not comparable across models of different loss scale.

The numbers. Table 10. Latent parent 1.00 ($\lambda_{\max} = 0.19$, $\lambda_{\min} = -718$), Llama 0.64, symbol engine 0.48, format-installed 0.00, blank 0.00 — again the order of the five curves. *Reading:* a parent about to be elicited sits on a saddle of the task loss, with directions that fall away steeply; a parent about to be taught sits in a bowl with no such shortcut.

M13 Answer carried by the state ★ — *swapping the state moves the answer vs. nothing moves*

Elicit vs. teach, in one sentence. Does the parent’s state at the answer position already carry the answer causally — swap it between two problems and the preference follows (elicit) — or does swapping it change nothing (teach)?

How it is measured. Distributed Alignment Search (Geiger et al. 2024), scored on preference rather than output so that it works on a parent that never answers. Take pairs of problems of equal prompt length. Run the parent on the base problem while overwriting its state after layer L at the answer position with the state from the source problem — either the whole state or only a learned k -dimensional subspace — and count how often the parent then prefers the source problem’s answer to the base problem’s. Controls: no swap, and a random subspace of the same size; 256 training and 128 test pairs, $L \in \{8, 12, 14, 15\}$, $k \in \{1, 4, 16, 64\}$.

The numbers. Table 10. Swapping the whole state raises the fraction of flipped preferences above the no-swap level by +0.39 (latent), +0.26 (Llama) and +0.30 (symbol engine), against +0.05 (format-installed) and +0.02 (blank); a learned 64-dimensional subspace recovers most of the full-swap effect (latent 0.64 against a random subspace’s 0.41 at layer 12). *Reading:* in every parent that can compute, the state at the answer position already holds the problem’s answer; in the teach parents it holds nothing that moves the answer.

M14 Number-reading heads ★ — *words already reach the arithmetic vs. they do not*

Elicit vs. teach, in one sentence. Do the parent’s attention heads already fetch the two numbers when the problem is posed in words (elicit), or not (teach) — and does having arithmetic suffice?

How it is measured. Desiderata Component Masking, as in M2, on the frozen parent and the natural-language prompts, with one change: since the parent never answers, a set of heads counts when copying its activations from a counterfactual problem (different first operand) moves the parent’s *preference* toward the counterfactual answer. Report the size of the learned head set and its flip rate against the ceiling of copying every head.

The numbers. Table 10. First-operand heads: latent parent 22 (flip 0.48, ceiling 0.67), Llama 23 (0.59 of 0.61); symbol engine, format-installed and blank: none. *Reading:* this is the metric that separates the symbol engine from the elicit parents. It has the arithmetic (the answer is in its state, previous metric) but no heads that carry the numbers from the words to it; the binding dose is what builds that interface, and only a parent with both is about to be elicited on this task.

M15 Operands present at the answer position ★ — *the numbers are there vs. weakly there*

Elicit vs. teach, in one sentence. Is the information the computation needs — the two numbers — already readable from the parent’s state at the moment it is about to answer?

How it is measured. Linear probe on the state at the answer position, layer by layer, about 2,000 addition problems (75/25 split, ridge with the regulariser chosen on held-out data). Target: helix features of each operand, $[y, \cos(2\pi y/T), \sin(2\pi y/T)]$ for $T \in \{2, 5, 10, 100\}$, the representation language models use for numbers (Kantamneni & Tegmark 2025). Report the best layer’s R^2 , averaged over the two operands.

The numbers. Table 10. Latent parent 0.88, Llama 0.86, symbol engine 0.90 against format-installed 0.53 and blank 0.49. *Reading:* every parent that can compute has gathered both numbers at the answer position; the teach parents hold them only partially. The answer chunk itself is not linearly readable beyond these copies in any parent: it is produced downstream and nonlinearly, where the hidden preference and the lens read it.

	latent	elicit Llama-3.2-1B	between symbol engine	teach format-installed	blank
hidden preference (nats)	2.31	1.00	0.44	0.06	0.02
curvature: negative share	1.00	0.64	0.48	0.00	0.00
answer state swap: flip gain over no swap	+0.39	+0.26	+0.30	+0.05	+0.02
number-reading heads (first operand)	22	23	0	0	0
operands present (R^2)	0.88	0.86	0.90	0.53	0.49
repeatable task circuit (split-half)	0.600	—	—	noise	noise

Table 10: Parent-only metrics on the five parents. The first two order all five as their learning curves do; the next two split the parents that can compute from those that cannot; number-reading heads alone separate the two elicit parents from the symbol engine, which has the arithmetic but not the interface to the words.